

test_smpl/ — the shared test suite

micro-os-plus-iii-smp/test_smpl/ holds **one copy of every test**, the table that says how each is configured, and the two runners that execute them. No architecture project keeps a copy of any of it.

That is the point of the directory. Before the unification the same twelve applications existed once per port, and the same `hw.sh` existed once per test per port — 48 files of about 200 near-identical lines. A fix to a test had to be made, and remembered, in every copy.

1. What is in it

```
test_smpl/
├─ CMakeLists.txt      exports micro-os-plus::test-common
├─ apps.cmake          WHICH tests exist and HOW each is configured
├─ run-qemu.sh         the emulator suite runner (every port)
├─ run-hw.sh           the hardware session runner (every port)
├─ common/
│   └─ include/
│       └─ test-console.hpp    console(), console_uart(), report_per_core()
│       └─ test-smp-boot.hpp   secondary-core bring-up and join helpers
│   └─ src/
│       └─ test-smp-boot.cpp
├─ smp_test0/ ... smp_test4/    main.cpp each
├─ smp-mat-test/ smp-mat-sdcard-test/
├─ smp-num-test/ smp-pipeline-test/ smp-pro-cons-test/
├─ sd_test/          main.cpp, flatfs_tool.py, verify.sh
├─ usb_test/         main.cpp, sink.cpp/.hpp, protocol.hpp,
                    host_xfer.py, send_file.py, README.md
```

The twelve applications

	what it proves
<code>smp_test0</code>	Board bring-up on one core: console, MMU, 1 ms tick, a thread sleeping on the system clock.
<code>smp_test1</code>	Two threads pinned to different cores exchanging messages; the secondaries join.
<code>smp_test2</code>	One worker pinned per CPU, all bumping a shared counter under a <code>μOS++</code> mutex. A final count of <code>ITER × OS_NCPU</code> means the kernel lock and the cacheable/shareable DRAM mapping are coherent — no lost updates.
<code>smp_test3</code>	Producer/consumer across cores.
<code>smp_test4</code>	The load balancer: workers with no affinity, checked for coverage of every core.
<code>smp-mat-test</code>	Parallel matrix work through a thread pool — the long, arithmetic-heavy one.
<code>smp-mat-sdcard-test</code>	The same, with results written to the card.
<code>smp-num-test</code>	Numerics plus storage.
<code>smp-pipeline-test</code>	A multi-stage pipeline across cores, with storage.
<code>smp-pro-cons-test</code>	Eleven kernel object types at once — mutex, memory pool, message queue, both semaphores, condition variable, event flags, timer, sysclock, yield/suspend/resume — and per-core work distribution.
<code>sd_test</code>	The SD/FatFs stack against a seeded volume.
<code>usb_test</code>	The DWC2 device stack. The only test QEMU cannot run at all, and the only one needing a host-side driver while it runs.

Five of them (`sd_test`, `smp-mat-sdcard-test`, `smp-num-test`, `smp-pipeline-test`, `usb_test`) link `micro-os-plus::devices`. A board whose silicon those drivers do not cover builds the other seven — see `UOS_TEST_APPS_NEED_SD` in `apps.cmake`.

What makes an application ISA-neutral

A test never contains inline assembly of its own and never names a peripheral. Everything machine-specific arrives through headers **the port supplies under names every port uses**:

```
uart.hpp    led.hpp    smp.hpp    timer_arm.hpp
exception_handler.hpp  hw_result.hpp
```

So `uart::uart1 << "..."` is a PL011 on the Pi and a DesignWare 16550 on the Lyra, and the test does not know which. Counts follow `OS_NCPU`, never a literal — the port sets it, and on the Lyra it is 3.

2. `apps.cmake` — the one table

The only file that knows which tests exist. Every architecture project reads it; none keeps its own list.

```
set (UOS_TEST_APPS          sd_test smp_test0 ... usb_test)
set (UOS_TEST_APPS_SMP_ONLY ...) # need OS_NCPU > 1
set (UOS_TEST_APPS_NEED_SD ...) # need micro-os-plus::devices

function (uos_test_app_defines _app _out) # the per-test -D knobs
```

Defines that are the same for **every** test — `TRACE`, `SEMIHOST`, the board macros — do **not** belong here. They depend on the toolchain and the board, which this file knows nothing about, so the architecture project adds them.

Adding a test is therefore: create `common/<name>/main.cpp`, add the name to `UOS_TEST_APPS`, and add it to `NEED_SD` / `SMP_ONLY` if it belongs there. Every port picks it up with no edit.

3. How a port consumes it

The architecture project adds the kernel repository as a subdirectory, which defines `micro-os-plus::test-common` and sets `UOS_TEST_COMMON` to `test_smpl/common`. Its own `test/CMakeLists.txt` then loops:

```
foreach (_app IN LISTS UOS_TEST_APPS)
  uos_test_app_defines ("${_app}" _app_defines)
  uos_add_test_app ("${_app}-hwd"
    APP "${_app}" # sources from test_smpl/common/<app>/
    NCPU ${_ncpu}
    LINKER_SCRIPT "${_variant_linker}"
    DEFINES ${_common_defines} ${_app_defines} ${_variant_defines}
    LIBRARIES ${_libs})
endforeach ()
```

`uos_add_test_app` globs every `.cpp` in the application directory, so a test that grows a second translation unit (`usb_test/sink.cpp`) needs no declaration. **No build ever spells out a test's source path.**

Each port builds two variants of each application:

variant	
-qemu	<code>QEMU_BUILD</code> . The image the emulator suite runs.
-hwd	<code>HW_BUILD</code> . Real silicon: the SD tests use the existing FAT32 boot partition instead of formatting a blank card.

4. Running them

Both runners live here and are **architecture-neutral** — the caller supplies the machine or the probe, the way `run-qemu.sh` takes its QEMU arguments.

```
run-qemu.sh <build-test-dir> <qemu-binary> <machine-args...>
```

Runs every `*-qemu` image and reports PASS/FAIL/SKIP from the `RESULT:` line each test prints. It also creates the SD images the card tests need — a seeded flatfs volume for `sd_test`, a blank image for the others.

```
cd micro-os-plus-iii-aarch64
../micro-os-plus-iii-smp/test_smpl/run-qemu.sh build/test \
    "$([ls ~/.local/xPacks/@xpack-dev-tools/qemu-arm/*/.content/bin/qemu-system-aarch64 | tail -1])" \
    -M raspi3b -smp 4
```

AArch32 on the Pi goes through the boot shim — see `building-aarch32-aarch64.md` §4.

```
run-hw.sh <build-test-dir> <app|list> [run-seconds]
```

Drives one `*-hwd` image on real silicon through OpenOCD, and reads the verdict from the semihosted console in OpenOCD's own log.

Pure OpenOCD: no GDB, no reset, and it never opens the serial device — so it cannot fight the terminal you keep on the console.

It refuses a suite. Every run is `load_image` into RAM over whatever the previous test left there, and neither supported board has a reset a script can drive. It takes exactly one test; you power-cycle between them. `list` is the default argument and prints what a build has, with each budget.

You do not normally call it directly. Each port wraps it in a ~40-line `test/hw.sh` supplying the port and board facts through the environment:

```
UOS_HW_CFG      UOS_HW_CFG_INIT  UOS_HW_NM      UOS_HW_READ ELF
UOS_HW_ENTRY    UOS_HW_SPIN_WORDS  UOS_HW_RESUME  UOS_HW_NCPU
UOS_HW_TARGET_FMT  UOS_HW_CORES    UOS_HW_PRELOAD  UOS_HW_ADAPTER_KHZ
```

```
cd micro-os-plus-iii-aarch32
test/hw.sh list                # the Pi
BOARD=luckfox-lyra test/hw.sh smp_test0  # the Lyra
```

A hardware budget is **not** the test's own duration. It is dominated by semihosting traps, and those scale with how much a test prints, not with how long it thinks it runs. `smp_test4` reaches its verdict at $t=9597$ ms of target time yet needs well over 120 s of wall clock, because its reporter emits about nine lines a second and each `<<` is a separate `SYS_WRITE0` — a debug halt and resume over the probe, roughly 0.15 s each.

5. The shared support headers

Two headers under `common/include/` hold what every test used to repeat.

`test-console.hpp` — `console()` and `console_uart()` take a lock, because `uart1` and semihosting are not re-entrant: without one, a `RESULT: PASS` line can be split by another core's output, and both runners grep for that exact string. `report_per_core()` prints a per-CPU tally array for as many cores as the port has.

`test-smp-boot.hpp` — the idle stacks, the idle body and `smp_install_boot_threads()` that every SMP test carried its own copy of, plus the join helpers `test_wait_secondaries()`, `test_secondaries_joined()` and `test_join_summary()`, and `cpu_slot()`.

Those last four exist because the tests used to name cores 1, 2 and 3 by hand and index per-core arrays with `port_cpu_id() & 3u`. Both are statements about a four-core BCM2837 rather than about the test: on a three-core board the first reads one past the end of `g_core_stage[OS_NCPU]`, and the second is a modulo only while the core count is a power of two, so core 2 lands in slot 0 and the distribution report is wrong **without failing**. See `aarch32-second-board.md` §4.

6. The rule this directory enforces

A test source exists exactly once. If you are about to copy a `main.cpp` into an architecture project, something is wrong with the port's headers instead — add the missing one under the name every port uses.

`tools/verify-kernel-compiles.sh` and the two QEMU suites are what keep that honest.